SoleSignal SP26-46 Capstone Design Project; Software Documentation

An embedded wearable sensor that converts discrete tap patterns into emergency SMS alerts.

Project Repository: https://github.com/anishshriram/SoleSignal

Advisor: Professor Minning Zhu, Rutgers University
Project Team: Om Patel, Seungmin Baik, Rishi Patel, Anish Shriram
{orp9, rp1202, sb1972, as3896}@scarletmail.rutgers.edu
Software Documentation authored by Anish Shriram

Copyright © 2026 Anish Shriram. All Rights Reserved.

This document and the software it describes are the intellectual property of the author. No part of this work may be used, modified, copied, distributed, or integrated into other projects without explicit written permission from the author. This project is currently developed as an academic work and is intended for potential commercial development in the future. This document is distributed solely for evaluation and demonstration purposes

Table of Contents

1 Acknowledgements
1.1 Artificial Intelligence Note
2 Executive Summary
2.1 The Problem
2.2 The Solution
2.3 How It Works
3 Software Minimum Viable Product (MVP) Definition
3.1 Frontend
3.2 Backend
3.3 Software MVP User Flow
3.4 Non-MVP Tasks
4 Business Glossary
4.1 Hardware Terms
4.2 Signal Processing Terms
4.3 Software / Backend Terms
5 Business Rules and Policies
5.1 Eligibility Policies
5.2 Alert Activation Policies
5.3 False Alarm Mitigation Policies
5.4 Data Privacy Policies
5.5 Safety and Misuse Prevention Policies
5.6 Logging and Auditing Policies
5.7 Operational Policies
6 Software Requirements
6.1 Functional Requirements
6.1.1 User Registration and Account Management
6.1.2 Sensor Pairing
6.1.3 Emergency Tap Pattern Detection
6.1.4 Alert Generation and Delivery
6.1.5 Emergency Contact Management
6.2 Non-Functional Requirements
6.2.1 Performance
6.2.2 Reliability
6.2.3 Security and Privacy
6.2.4 Usability
6.3 System and Operational Constraints
6.4 Out of Scope Requirements

6.5 Assumptions
7 Software System Architecture
7.1 Architecture Scope
7.2 Basic Architectural Components
7.3 Key Application Flow
7.4 Data Sources, System Integration, and External APIs
7.5 Technology Stack
7.6 Backend File Structure
7.7 Mobile App Screens
8 Use Cases
8.1 UC-1: Register User Account
8.2 UC-2: Pair Sensor with User Account
8.3 UC-3: Manage User Profile
8.4 UC-4: Manage Emergency Contacts
8.5 UC-5: Detect Emergency Tap Pattern
8.6 UC-6: Send Emergency Alert
9 Domain Model
9.1 User
9.2 Sensor
9.3 Emergency Contact
9.4 Alert
9.5 Entity Relationships
10 Database Schema
10.1 users
10.2 sensors
10.3 contacts
10.4 alerts
11 API Specification and REST API Design
11.1 Endpoint Overview Table
11.2 registerUser()
11.3 loginUser()
11.4 updateUserProfile()
11.5 logoutUser()
11.6 pairSensor()
11.7 getSensorStatus()
11.8 getMySensor()
11.9 unpairSensor()
11.10 addContact()
11.11 getContacts()
11.12 updateContact()

11.13 deleteContact()
11.14 sendAlert()
11.15 getAlertStatus()
12 Server Side Routing of HTTP Requests
12.1 Auth Middleware
12.2 registerUser()
12.3 loginUser()
12.4 updateUserProfile()
12.5 logoutUser()
12.6 pairSensor()
12.7 getSensorStatus()
12.8 getMySensor()
12.9 unpairSensor()
12.10 addContact()
12.11 getContacts()
12.12 updateContact()
12.13 deleteContact()
12.14 sendAlert()
12.15 getAlertStatus()
13 Authentication and Session Management
13.1 Token Generation
13.2 Token Storage
13.3 Token Usage
13.4 Token Expiry and Renewal
13.5 Protected vs. Unprotected Endpoints
13.6 Ownership Enforcement
13.7 Logout Procedure

Acknowledgements

Professor Minning Zhu for his time, advice, and constant push to keep on innovating, even when I didn't think possible!

My SP-46 Groupmates, Om Patel, Rishi Patel, and Seungmin Baik for their work on the hardware and signal processing components of this project, as without it SoleSignal could not reach its potential.

The Rutgers University School of Engineering, specifically the Department of Electrical and Computer Engineering for its four years of critical education, making this project possible. More recently, its access to advisors, materials, and lab equipment so that we could bring our ideas into a physical manifestation.

Most importantly, thank you Amma, Appa, and Nilay for supporting me in my highest and lowest moments. I love you all!

Artificial Intelligence Note:

While AI tools were used to assist in various aspects of SoleSignal, all technical content, implementation decisions, and system design reflect the work of the author and the SP-46 team.

The models used, and their use cases are as follows:

1. OpenAI's ChatGPT GPT-5.4 was used to optimize firmware code and make adjustments to the CNN Algorithm
2. Anthropic's Claude Sonnet 4.6 was used to assist conversion from a web-based application to a mobile application and update a central changelog for continuous documentation purposes
3. Google's Gemini 3.1 Flash was used to assist in figure generation

We are extremely excited to present our hard work today, and we hope you enjoy as well as learn something new!

Executive Summary

The Problem

There are various situations where the act of reaching for a phone itself is dangerous. A person in the presence of an active threat, whether that is a domestic abuser, an abductor, or any aggressor monitoring their behavior, cannot always safely call for help or make any visible gesture toward a device. Existing personal safety tools, from panic buttons to emergency features on smartphones, share a common flaw: they require a visible and deliberate action that can be intercepted or punished. For the populations most at risk of this kind of threat, the tool itself can become a liability. This technology can be extended to everything from a construction worker attempting to signal danger in a high noise environment to a university professor clicking through slides without having to use a clicker.

The Solution

SoleSignal is a sensor embedded in footwear that detects a specific tap pattern performed discreetly with the foot and translates it into an emergency SMS alert sent to pre-designated contacts. The hardware runs on an ESP32-C3 microcontroller and uses a TFLite Inference machine learning model trained to distinguish international SOS tap sequences from normal walking, running, and incidental movement. Once the system detects a credible match, it communicates with a paired smartphone application over Bluetooth Low Energy, which sends an alert package along with the user’s current location. A haptic motor in the device confirms activation to the wearer through vibration, without producing any sound or visible signal,

How It Works

The sensor samples motion data continuously through two analog input channels consisting of force sensitive resistors. Each sample is normalized and fed into a rolling 50 sample buffer. When the buffer fills, a TFLite Inference model evaluates the sequence and returns a confidence score. A voting system requires the model to return a high confidence result across three consecutive windows before triggering an alert, a design choice that eliminates false positives from brief accidental gestures. Once activated, the system sends a SMS through the paired application containing the user’s identity, location, and a distress flag. The message reaches designated emergency contacts within five seconds of a confirmed pattern. Presently, SMS capability is achieved using Textbelt's pay-as-you-go API.

Software Minimum Viable Product (MVP) Definition

Frontend:

The frontend application will do, at minimum, the following:

1. Register with the user
2. Pair with the sensor
3. Receive alerts/data from the sensor and forward an alert package to a chosen emergency contact
    a. SMS notification with a distress message, GPS location (or "location unavailable" if GPS is inaccessible), phone number, and other basic information (timestamp, sensor ID)

Backend:

The backend application will be constructed as follows:

1. The backend is implemented as a REST API using Node.js 20, Express v5, and TypeScript 6
2. The database is PostgreSQL 14, managed via Prisma ORM (raw SQL is not used directly)
3. The mobile application is built with React Native 0.84 (bare CLI), targeting iOS only for the MVP
4. Textbelt API will be used for SMS delivery, replacing Twilio (A2P 10DLC and toll-free verification blocked Textbelt for US numbers)

Software MVP User Flow:

1. User downloads the application
2. User registers for an account
3. User pairs their sensor with their account
4. User adds at least one emergency contact
5. When the user applies their specific tap pattern, the app sends an alert package

Non MVP Tasks:

- Law enforcement integrations
- Tamper detection
- Multi-sensor identity binding
- False alarm secondary confirmations
- Data retention policies
- Encrypted cloud logging
- Automatic audit trails
- Footstep filtering ML models

Business Glossary

Hardware Terms

Sensor: The SoleSignal shoe insert device containing FSR pressure sensors and a microcontroller that detects foot tap patterns.
Force Sensitive Resistor (FSR): A pressure sensor embedded in the shoe insert that converts applied foot pressure into an analog voltage signal.
Activation Window: The defined time period within which all taps in a valid emergency pattern must occur.
Tap Threshold: The minimum force level a tap must exceed to be counted as part of a valid pattern.
Haptic Feedback: Vibration confirmation delivered by the user's phone after an emergency alert is dispatched.

Signal Processing Terms

Tap Pattern: The specific sequence of taps (count, force, and timing) that triggers an emergency alert.
CNN (Convolutional Neural Network): The machine learning model that classifies pressure sensor data to distinguish intentional tap gestures from normal walking or running.
Voting System: The false positive prevention mechanism requiring 3 consecutive high confidence CNN windows before an SOS is triggered.
EMA (Exponential Moving Average) Filter: Exponential Moving Average filter applied to raw sensor data to suppress noise.

Software / Backend Terms

Alert Package: The data bundle assembled and sent to an emergency contact, including user identity, GPS location, timestamp, and sensor ID.
Cancel Window: The 10 second countdown after tap detection during which the user can cancel the alert before it is dispatched.
JWT (JSON Web Token): The authentication token issued at login, used to identify and authorize the user on all protected API requests.
Textbelt: The third-party SMS API used to deliver emergency alert messages to designated contacts.
Nominatim: The OpenStreetMap reverse geocoding service used to convert GPS coordinates into a traditional address for the SMS message.
Prisma: The database management layer that handles schema definition, migrations, and queries for the PostgreSQL database.
BLE (Bluetooth Low Energy): The wireless communication protocol used to connect the shoe sensor to the paired smartphone.

Business Rules and Policies

Eligibility Policies

1. Users must register an account before activating a sensor.
2. Identity verification is required for the primary user.
3. Each shoe sensor can be linked to only one registered user at a time.

Alert Activation Policies

1. Emergency tap patterns must be deliberate, as defined by
    a. A specific tap count,
    b. A minimum tap force threshold,
    c. Timing between taps within a defined window.
2. Any tap sequences not meeting the defined emergency pattern must be rejected.
3. The full pattern must occur within a defined activation window.
4. Alerts cannot be triggered while the device is in calibration or maintenance mode.

False Alarm Mitigation Policies

1. Low confidence requires secondary confirmation signals.
2. Users shall be provided with a cancel alert window unless disabled for safety reasons.
3. Sensors must undergo routine calibration at specified intervals to minimize false activation.

Data Privacy Policies

1. Raw motion data may only be temporarily stored for tap pattern verification.
2. Personally identifiable information collected may include:
    a. User identity,
    b. Emergency contact information,
    c. GPS location at time of alert,
    d. Emergency activation timestamp.
3. Continuous location tracking is prohibited unless the user opts in.
4. Users may request deletion of historical alert data unless legal retention requirements apply.

Safety and Misuse Prevention Policies

1. The sensor must support tamper detection.
2. Users may not assign false identities or impersonate others within the system.
3. Emergency alerts must not be triggered by normal activities (ex. running, walking, exercising).
4. The sensor must not interfere with user mobility.

Logging and Auditing Policies

1. All emergency activation must be logged with sensor ID, user ID, timestamp, confidence level, and location.
2. Log data must be immutable and protected from unauthorized modification.
3. Users must be notified after an alert is processed unless delayed notification is necessary for user safety.
4. System operators must comply with valid law enforcement or legal audit requests.

Operational Policies

1. Hardware must meet durability and reliability requirements defined by the organization.
2. All components must comply with safety regulations.
3. Software and firmware updates must be digitally signed and verified before installation.
4. Devices must fail safely (loss of battery, signal, or connectivity shall not trigger false alerts).

Software Requirements

1. FUNCTIONAL REQUIREMENTS

1.1 User Registration and Account Management

FR-1. The application shall allow a user to create an account using email or phone number.
FR-2. The system shall verify the user's identity via email/SMS verification.
FR-3. The system shall allow the user to log in and log out of their account.
FR-4. The system shall allow users to manage (edit/remove) their personal profile information.

1.2 Sensor Pairing

FR-5. The application shall pair with one SoleSignal sensor via Bluetooth.
FR-6. The system shall enforce that one sensor is linked to only one user account at a time.
FR-7. The system shall display a confirmation when the sensor is successfully paired.
FR-8. The system shall notify the user if the sensor disconnects unexpectedly.

1.3 Emergency Tap Pattern Detection

FR-9. The sensor shall send tap-pattern data (timestamp, force, count) to the application.
FR-10. The application shall validate the incoming pattern using predefined rules:

- Specific tap count
- Minimum tap-force threshold
- Tap timing within a defined activation window

FR-11. The application shall reject incomplete or invalid patterns.
FR-12. The system shall ignore tap patterns while the sensor is in calibration or maintenance mode.

1.4 Alert Generation and Delivery

FR-13. When a valid emergency pattern is detected, the app shall generate an alert package that includes:

- User identity
- GPS location
- Timestamp
- Sensor ID
- Optional user message (future feature; not MVP)

FR-14. The system shall send the alert package to the user's designated emergency contact using:

- Textbelt API
  <u>OR</u>
- The device's native SMS system

FR-15. The system shall confirm to the user (via UI) that an alert has been sent.

FR-16. If GPS is unavailable, the system shall still send the alert with “location unavailable” in the message.

1.5 Emergency Contact Management

FR-17. The application shall allow the user to add at least one emergency contact.
FR-18. The user shall be able to edit or delete emergency contact information.
FR-19. The system shall validate that a contact phone number is reachable (basic format validation + SMS capability check).

2. NON-FUNCTIONAL REQUIREMENTS

2.1 Performance

NFR-1. The system shall deliver an alert message within 5 seconds after pattern recognition, assuming cellular service is available.
NFR-2. Sensor to app Bluetooth reconnection shall occur within 3 seconds after temporary disconnection.
NFR-3. Tap pattern processing shall complete within 150 ms of receiving sensor data.

2.2 Reliability

NFR-4. The system shall function correctly under walking, running, and routine motion without triggering false alerts.
NFR-5. The app shall retry SMS delivery up to 3 times if Textbelt or network failures occur.
NFR-6. The system shall not trigger alerts due to sensor power loss or app crash (“fail-safe” requirement).

2.3 Security and Privacy

NFR-7. Continuous GPS tracking shall be disabled by default.
NFR-8. Raw motion data shall only be stored temporarily in RAM and must not persist on device storage.
NFR-9. All user accounts, contacts, and sensitive fields shall be stored securely using platform-provided encryption.
NFR-10. Emergency alert data shall not be transmitted to law enforcement.

2.4 Usability

NFR-11. The pairing process shall complete in under 30 seconds.
NFR-12. Emergency contacts must be added during onboarding to reduce user error.
NFR-13. All error states (e.g., sensor offline, GPS disabled) must be communicated clearly to the user.

3. SYSTEM AND OPERATIONAL CONSTRAINTS

C-1. Each sensor may be linked to only one registered user.
C-2. Emergency tap patterns must be deliberate and meet the force/timing requirements.
C-3. Alerts cannot be triggered during calibration or maintenance.
C-4. No continuous GPS tracking unless explicitly enabled.
C-5. The application must not collect or store more data than required for an alert.
C-6. The app must not impersonate users or generate alerts under a false identity.
C-7. The app must not trigger alerts from normal motion patterns.
C-8. The system must send alerts only to the user's chosen contact.

4. OUT OF SCOPE REQUIREMENTS

OS-1. Law enforcement integrations
OS-2. Tamper detection and integrity checks
OS-3. Multi-sensor identity binding
OS-4. Secondary confirmation signals (false alarm mitigation)
OS-5. Automatic audit trails and centralized logs
OS-6. Encrypted cloud storage for historical alerts
OS-7. Multi-contact redundancy (multi-recipient alerting)
OS-8. Sensor calibration UI
OS-9. Footstep filtering ML models
OS-10. Battery optimization and sensor-wearable power management
OS-11. Non-SMS alert channels (WhatsApp, email, push notifications)

5. ASSUMPTIONS

A-1. Users have a smartphone with Bluetooth and cellular capability.
A-2. The phone OS allows background processing for receiving sensor data.
A-3. Textbelt or SMS functions are accessible in the user’s region.
A-4. The shoe sensor hardware is already functional and able to transmit tap data.

Software System Architecture

Architecture Scope

This System Architecture describes the mobile application software only for the SoleSignal MVP. Hardware, firmware, and signal processing algorithms are considered external systems and are not designed here.

Basic Architectural Components

1. User Account and Authentication Module

FR-1, FR-2, FR-3, FR-4

This module lets users sign up, verify their identity, log in, and manage their profile, which is crucial so that emergency contacts/services know exactly who is making an alert, and it also helps the data stay organized.

2. Sensor Pairing and Bluetooth Module

FR-5, FR-6, FR-7, FR-8

This module deals with ensuring the sensor itself is adequately connected to the application. It will ensure that the sensor is able to be paired, ensures a secure connection, and notifies the user if there is a sensor disconnection.

3. Emergency Management Module

FR-9, FR-10, FR-11, FR-12, FR-17, FR-18, FR-19, NFR-12

There are a couple of parts to be considered for this module. It needs to contain a 'listener' which will pay attention to the signal processing algorithm outputs, and if the algorithm provides some output which signifies distress, then the application will take next steps. It will however ignore triggers when told the sensor is in calibration/maintenance mode. This module will also manage emergency contacts, allowing the user to input, delete, and otherwise manage their own list of emergency contacts.

4. Alert Module

FR-12, FR-14, FR-15, FR-16, NFR-1, NFR-5

This module is responsible for gathering all the information needed for an alert, and sending it. It will fetch location data only at the time of an alert (no continuous or background tracking), and will assemble a full package including user identity, location, a timestamp, and the sensor ID. It will use the Textbelt API to send this package to a chosen emergency contact, retry if the message failed to send, and send a confirmation notification to the user.

5. Storage and Logging Module

NFR-8, NFR-9

This module will work with the database/backend architecture to store important information, as well as keep a log for auditing purposes. It will securely store login/session info, emergency contacts paired, and sensor IDs. To comply with the NFRs above, it will not store raw motion data from the sensor, or continuous GPS information.

6. UI/UX Module

FR-7, FR-8, FR-15, NFR-11, NFR-13
This module will help guide onboarding and use, providing a clear flow from registration, to pairing, to adding contacts, etc. It will also display information such as sensor connection status, alert sent confirmation, and clear error messages

Key Application Flow

1. App receives emergency trigger from paired sensor.
2. App requests current GPS location.
3. App builds the alert package.
4. App sends the alert via SMS.
5. App confirms alert delivery to the user.

Data Sources, System Integration, and Usage of External APIs

The SoleSignal mobile application integrates internal system data with a limited number of external services to support emergency alert delivery. The MVP architecture is intentionally minimal and relies primarily on on-device capabilities, with a single external API used for SMS delivery.

There is a series of internal data sources that this application needs, including:

1. User Account Data
    a. Includes verified user identity and session information used to associate alerts with a specific user.
2. Emergency Contact Data
    a. Stores emergency contact phone numbers used for alert delivery.
3. Sensor Metadata
    a. Includes paired sensor identifiers and connection status.
4. Location Data
    a. GPS location is retrieved only at the time an emergency alert is triggered. Continuous location tracking is not performed.

The only external API to be used will be the Textbelt API, a simple HTTP-based SMS service. SoleSignal will use it to deliver emergency alert messages to designated contacts.
Data Sent:

- User identification (or display name)

- Emergency alert message
- GPS location (if available)
- Timestamp of activation
- Sensor identifier

Integration Flow:

1. When a valid emergency trigger is received, the application assembles an alert package.
2. The alert package is sent to Textbelt via a secure HTTP request.
3. Textbelt delivers the SMS message to the emergency contact's phone number.
4. The application confirms successful delivery or retries the request if delivery fails.

Error Handling and Safeguards:

- Message Delivery Failures
    - If an SMS fails to send via Textbelt, the system retries delivery up to the configured limit. If delivery cannot be completed, the user is notified of the failure.
- Location Unavailability
    - If GPS data cannot be obtained, the alert is still sent with a "location unavailable" indicator.
- Sensor or Network Errors
    - Sensor disconnections and network issues are clearly communicated to the user through the application interface.

Technology Stack

Backend:

- Runtime: Node.js 20
- Framework: Express v5
- Language: TypeScript 6
- ORM: Prisma
- Database: PostgreSQL 14
- HTTP Logger: Morgan
- Password Hashing: bcryptjs (minimum 8 characters enforced)
- JWT Library: jsonwebtoken (24h expiry, HS256 algorithm)
- SMS: Textbelt API

Mobile:

- Framework: React Native 0.84
- Platform: iOS only
- BLE Library: react-native-ble-plx
- Secure Storage: react-native-keychain
- GPS: @react-native-community/geolocation
- Navigation: @react-navigation/native-stack

- HTTP Client: Axios

Backend File Structure

```
backend/
  server.ts                 – Entry point; calls app.listen()
  app.ts                    – Express app definition
  middleware/
    auth.ts                 – JWT validation middleware
  routes/
    users.ts
    sensors.ts
    contacts.ts
    alerts.ts
  types/
    express.d.ts            – Extends Express request to include req.user.user_id
  prisma/
    schema.prisma           – Prisma schema (source of truth for DB structure)
    migrations/             – Migration history managed by Prisma
```

Mobile App Screens

The UI/UX module contains the following screens:

Screen	Route Name	Description
RegisterScreen	Register	Email, name, phone, password form; calls POST /users/register
LoginScreen	Login	Email, password form; calls POST /users/login
HomeScreen	Home	Sensor status dot, event log, send alert button, reconnect/unpair actions
PairingScreen	Pairing	BLE Scan, connect; calls POST /sensors/pair
ContactsScreen	Contacts	List, add, edit, delete contacts
AlertSentScreen	AlertSent	Confirmation screen showing alert ID after dispatch

Onboarding flow: Register – Pair – Contacts – Home (enforced by navigation)

Use Cases

Use Case UC-1: Register User Account

Related Requirements: *FR-1, FR-2, NFR-9, NFR-12, NFR-13, C-6*
Initiating Actor: User
Actor's Goal: To create a new SoleSignal account so that they can pair their sensor and configure emergency alerts.
Participating Actors: System
Preconditions:

- The user has downloaded the SoleSignal application.
- The user has a smartphone with cellular capability.
- The user does not already have a registered account.

Minimal Guarantees:

- If registration fails, no account is created and the user is notified of the failure.
- No partial or incomplete account data is stored.

Success Guarantees:

- A verified user account is successfully created and stored securely.
- The user is directed to the next onboarding step.

Main Success Scenario:

1. The user opens the SoleSignal application.
2. The user selects "Create Account."
3. The user enters their name, email address or phone number, and creates a password.
4. The system validates the input and creates the account.
5. The user is directed to the sensor pairing screen.

Alternate Scenarios and Contingences:

- Invalid email/phone format: *The system notifies the user and prompts them to correct the input before proceeding.*
- Account already exists: *The system notifies the user and redirects them to the login screen.*

Use Case UC-2: Pair Sensor with User Account

Related Requirements: *FR-5, FR-6, FR-7, FR-8, NFR-2, NFR-11, NFR-13, C-1*
Initiating Actor: User
Actor's Goal: To pair their SoleSignal shoe sensor with their account so that the application can receive tap pattern data.
Participating Actors: System, SoleSignal Sensor
Preconditions:

- The user has a registered and verified account.
- The user is logged into the application.

- The user has a SoleSignal sensor and Bluetooth is enabled on their smartphone.

Minimal Guarantees:

- If pairing fails, no sensor is linked to the account and the user is notified.
- The system does not link a sensor that is already paired to another account.

Success Guarantees:

- The sensor is successfully linked to the user's account.
- The user receives a confirmation that pairing was successful.

Main Success Scenario:

1. The user navigates to the sensor pairing screen.
2. The user powers on their SoleSignal sensor.
3. The application scans for available Bluetooth devices.
4. The user selects their sensor from the list of discovered devices.
5. The system verifies the sensor is not already linked to another account.
6. The system establishes a Bluetooth connection with the sensor.
7. The system displays a confirmation that the sensor has been successfully paired.
8. The user is directed to the next onboarding step.

Alternate Scenarios and Contingences:

- Sensor not discovered: *The system notifies the user and prompts them to ensure the sensor is powered on and within range.*
- Sensor already linked to another account: *The system denies pairing and notifies the user.*
- Bluetooth connection drops during pairing: *The system notifies the user and prompts them to retry.*
- Pairing takes longer than 30 seconds: *The system times out, notifies the user, and offers the option to retry.*

Use Case UC-3: Manage User Profile

Related Requirements: *FR-3, FR-4, NFR-9, NFR-13, C-6*

Initiating Actor: User

Actor’s Goal: To log in, log out, or update their personal profile information within the SoleSignal application.

Participating Actors: System

Preconditions:

- The user has a registered and verified account.
- The application is installed and operational.

Minimal Guarantees:

- If a profile update fails, the original information is preserved and the user is notified.
- Unauthorized users cannot access or manage another user’s profile.

Success Guarantees:

- The user is successfully logged in or logged out.

- Any profile changes are saved securely and reflected in the application.

Main Success Scenario:

1. The user opens the SoleSignal application.
2. The user enters their credentials and logs in.
3. The user navigates to their profile settings.
4. The user edits their personal information (e.g. name, email, phone number).
5. The system validates the updated information.
6. The system saves the changes securely.
7. The application reflects the updated profile information.
8. When finished, the user logs out of the application.

Alternate Scenarios and Contingences:

- Invalid credentials on login: *The system notifies the user and prompts them to reenter their credentials.*
- Invalid or improperly formatted profile update: *The system notifies the user and prompts them to correct the input.*
- Profile update fails due to a system error: *The system preserves the original data and notifies the user of the failure and then prompts them to try again.*

Use Case UC-4: Manage Emergency Contacts

Related Requirements: *FR-17, FR-18, FR-19, NFR-9, NFR-12, NFR-13*

Initiating Actor: User

Actor's Goal: To add, edit, or remove emergency contacts so that the system knows who to notify in the event of an emergency.

Participating Actors: System

Preconditions:

- The user has a registered and verified account.
- The user is logged into the application.

Minimal Guarantees:

- If a contact cannot be saved, the user is notified and no incomplete contact information is stored.
- If a contact is deleted, it is fully removed from the system.

Success Guarantees:

- At least one valid emergency contact is saved to the user's account.
- The user can view, edit, and remove the emergency contacts at any time.

Main Success Scenario:

1. The user navigates to the emergency contacts screen.
2. The user selects "Add Emergency Contact."
3. The user enters the contact's name and phone number.
4. The system validates that the phone number is correctly formatted and SMS capable.
5. The system saves the contact to the user's account.

6. The application displays the newly added contact in the user's contact list.

Alternate Scenarios and Contingences:

- Invalid phone number format: *The system notifies the user and prompts them to correct the input before saving.*
- Phone number fails SMS capability check: *The system warns the user that the contact may not be reachable and prompts them to verify the number.*
- User attempts to proceed without adding a contact: *The system requires at least one contact to be added before the onboarding process can be completed.*
- User edits an existing contact: *The system validates and saves the updated information, replacing the previous entry.*
- User deletes a contact: *The system removes the contact and updates the contact list accordingly.*

Use Case UC-5: Detect Emergency Tap Pattern

Related Requirements: *FR-9, FR-10, FR-11, FR-12, NFR-3, NFR-4, C-2, C-3, C-7*

Initiating Actor: SoleSignal Sensor

Actor's Goal: To transmit tap pattern data to the application so that the system can determine whether a valid emergency pattern has been activated.

Participating Actors: User, System

Preconditions:

- The user has a registered and verified account.
- The user is logged into the application.
- The sensor is paired and connected to the application via Bluetooth.
- The sensor is not in calibration or maintenance mode.

Minimal Guarantees:

- If the tap pattern is invalid or incomplete, no alert is triggered and the system returns to idle monitoring.
- Normal activity such as walking or running does not trigger a false activation.

Success Guarantees:

- A valid emergency tap pattern is recognized and passed to the alert module for further action.
- The user receives haptic feedback from the phone confirming the alert has been dispatched.

Main Success Scenario:

1. The sensor continuously monitors foot pressure via the FSR sensors.
2. The user performs a deliberate tap pattern on the sensor.
3. The sensor transmits tap data including timestamp, force, and count to the application.
4. The application validates the incoming pattern against predefined rules including tap count, minimum force threshold, and timing window.
5. The pattern is confirmed as valid.

6. The phone vibrates to notify the user that the alert countdown has begun.
7. The system passes the validated trigger to the alert module.

Alternate Scenarios and Contingences:

- Tap pattern does not meet force threshold: *The system rejects the pattern and returns to idle monitoring.*
- Tap pattern does not complete within the activation window: *The system resets and returns to idle monitoring.*
- Tap count does not match the required pattern: *The system rejects the input and returns to idle monitoring.*
- Sensor is in calibration or maintenance mode: *The system ignores all tap inputs until the mode is exited.*
- Sensor disconnects mid-pattern: *The system discards the incomplete pattern, notifies the user of the disconnection, and returns to idle monitoring.*

Use Case UC-6: Send Emergency Alert

Related Requirements: *FR-13, FR-14, FR-15, FR-16, NFR-1, NFR-5, NFR-7, NFR-8, C-4, C-5, C-8*

Initiating Actor: System

Actor's Goal: To assemble and deliver an emergency alert package to the user's designated emergency contact as quickly and reliably as possible.

Participating Actors: User, Emergency Contact, Textbelt API

Preconditions:

- A valid emergency tap pattern has been detected and passed from the alert module.
- The user has at least one emergency contact saved to their account.
- The sensor is not in calibration or maintenance mode.

Minimal Guarantees:

- If the alert fails to send, the user is notified of the failure.
- No continuous GPS data is collected or stored as a result of this process.
- Raw motion data is not stored beyond tap pattern verification.

Success Guarantees:

- The alert package is successfully delivered to the user's emergency contact within 5 seconds.
- The user receives confirmation in the application that the alert was sent.

Main Success Scenario:

1. The system receives a validated emergency trigger from the tap detection module.
2. The system retrieves the user's identity, sensor ID, and emergency contact information.
3. The system requests the user's current GPS location.
4. The system assembles the alert package including user identity, GPS location, timestamp, and sensor ID.
5. The system transmits the alert package to the emergency contact via the Textbelt API.

6. Textbelt delivers the SMS message to the emergency contact's phone number.
7. The system receives confirmation of successful delivery.
8. The application notifies the user that the alert has been sent.

Alternate Scenarios and Contingences:

- GPS is unavailable: *The system still assembles and sends the alert package with a "location unavailable" indicator in place of GPS data.*
- Textbelt delivery fails: *The system retries delivery up to 3 times before notifying the user of the failure.*
- Network is unavailable: *The system notifies the user that the alert could not be sent and continues retrying until connectivity is restored or the retry limit is reached.*
- Emergency contact phone number is unreachable: *The system notifies the user that delivery failed after exhausting all retry attempts.*

Domain Model

The SoleSignal domain model defines the core entities, attributes, and relationships that make up the data layer of the application. It serves as the structural foundation for the database schema, API design, and overall system architecture. The MVP architecture is intentionally minimal, but it is still crucial to document how key entities relate towards each other, and what backend services must operate on them.

This section will describe the core entities in the system, including:

- User
- Sensor
- Emergency Contact
- Alert

And will describe the following:

- The relationships between these entities
- The various services the backend processes
    - Alert Delivery
    - Bluetooth and Sensor Management
    - Location Retrieval
    - User Authentication
- And a high level description of how data flows within the system

USER

Represents a registered individual who has created a SoleSignal account. All other entities are associated with a specific verified user.

Attribute	Description
id	Unique identifier for the user
name	Full name of the user
email	Email address used for registration and verification
phone_number	Phone number used for registration and verification
password_hash	Securely stored password
is_verified	Boolean flag indicating whether the user account has been registered. Note: email/SMS verification is not implemented in the MVP; this field defaults to false and is not updated.
last_login	Timestamp of the user's most recent login
created_at	Timestamp when the account was created

SENSOR

Represents a physical SoleSignal shoe insert device that has been registered in the system. Each sensor can only be linked to one user at a time, and a user can only have one paired sensor at a time.

Attribute	Description
id	Unique identifier for the sensor
sensor_id	Hardware identifier assigned to the physical device
user_id	Links the sensor to a specific registered user
is_paired	Boolean flag indicating whether the sensor is currently paired to a user account
is_calibrating	Boolean flag indicating whether the sensor is currently in calibration or maintenance mode
last_connected	Timestamp of the most recent successful Bluetooth connection
created_at	Timestamp of when the sensor was registered in the system

EMERGENCY CONTACT

Represents a person designated by the user to receive an SMS alert in the event of an emergency activation. At least one emergency contact must be saved to the user's account before the system becomes operational.

Attribute	Description
id	Unique identifier for the emergency contact
user_id	Links the emergency contact to a specific registered user
name	Full name of the emergency contact
phone_number	Phone number of the emergency contact used for SMS alert delivery
is_valid	Boolean flag indicating whether the phone number has passed format validation and SMS capability check
created_at	Timestamp when the emergency contact was added

ALERT

Represents an emergency alert event. The Alert entity stores all relevant data about the activation event, including the user's location at the time of the alert and the delivery status of the SMS message.

Attribute	Description
id	Unique identifier for the alert

user_id	Links the alert to the user who triggered it
sensor_id	Links the alert to the sensor that detected the tap pattern
contact_id	Links the alert to the emergency contact who was notified
gps_latitude	Latitude coordinate of the user's location at the time of activation (stored as Float / DOUBLE PRECISION in the database)
gps_longitude	Longitude coordinate of the user's location at the time of activation (stored as Float / DOUBLE PRECISION in the database)
location_available	Boolean flag indicating whether the GPS data was successfully retrieved
timestamp	Time at which the emergency alert was triggered
delivery_status	Indicates whether the SMS was successfully delivered, failed, or is pending
retry_count	Number of SMS delivery attempts made by the system

Entity Relationships

User → Sensor (1:1)

A User can have one Sensor paired to their account, and a Sensor can only be linked to one User at a time.

Implementation: sensor.user_id is a foreign key referencing user.id

Example:

User ID	User Name	Sensor ID
001	John Doe	S001
002	Jane Doe	S002

User → Emergency Contact (1:M)

A User can have one or more Emergency Contacts saved to their account but each Emergency Contact belongs to only one User.

Implementation: contact.user_id is a foreign key referencing user.id

Example:

User ID	User Name	Contact ID	Contact Name
001	John Doe	C001	Jane Doe
001	John Doe	C002	Bob Smith
002	Jane Doe	C003	John Doe

User → Alert (1:M)

A User can have many Alerts associated with their account over time, but each Alert belongs to only one User.
Implementation: alert.user_id is a foreign key referencing user.id
Example:

User ID	User Name	Alert ID	Timestamp
001	John Doe	A001	2026-01-01 12:35:00
001	John Doe	A002	2026-01-01 14:42:00
002	Jane Doe	A003	2026-01-02 13:25:00

Sensor → Alert (1:M)

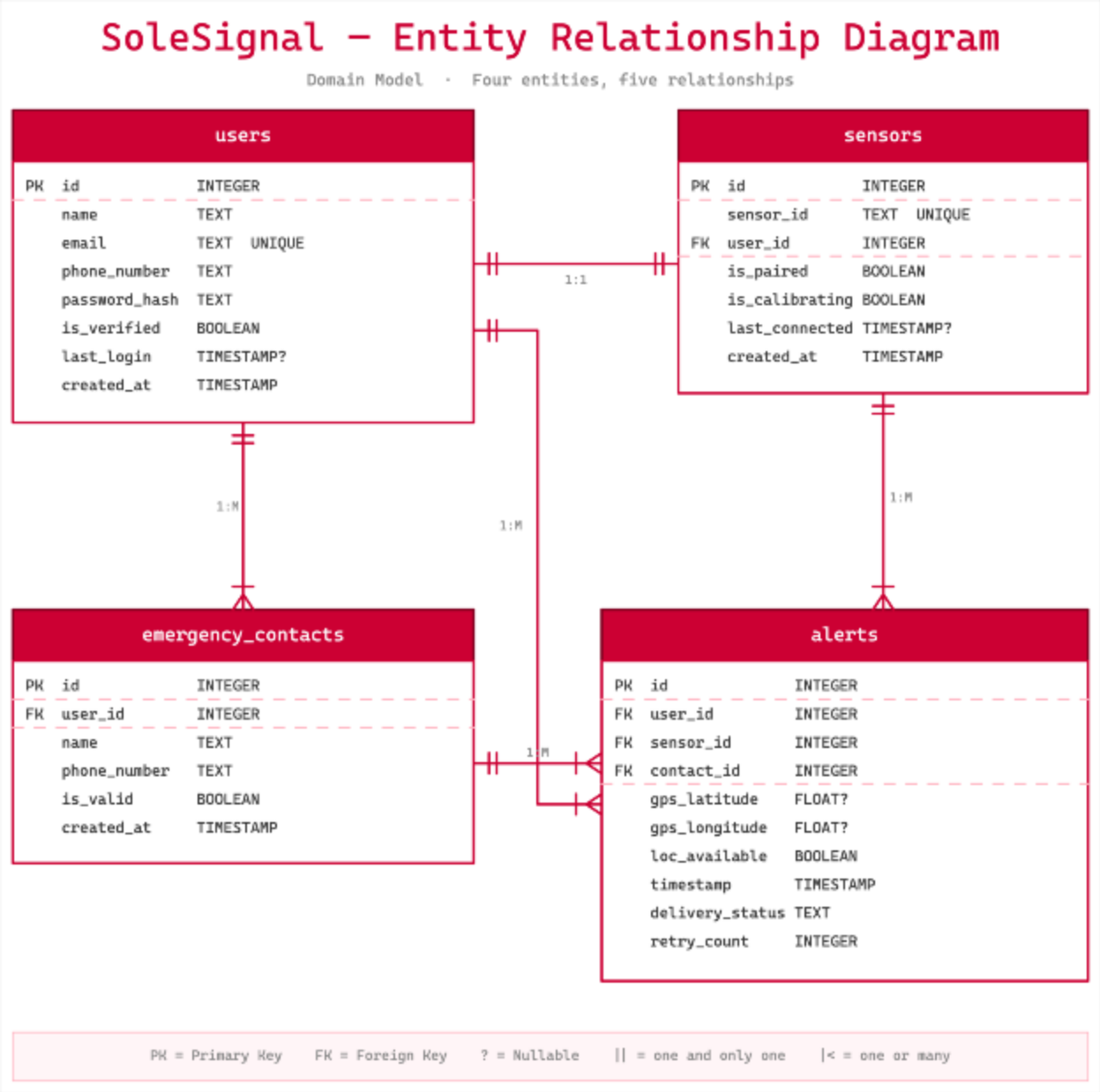
A Sensor can be associated with many Alerts over time, but each Alert is tied to only one Sensor that detected the tap pattern.
Implementation: alert.sensor_id is a foreign key referencing sensor.id
Example:

Sensor ID	Alert ID	Timestamp
S001	A001	2026-01-01 12:35:00
S001	A002	2026-01-01 14:42:00
S002	A003	2026-01-02 13:25:00

Emergency Contact → Alert (1:M)

An Emergency Contact can be associated with many Alerts over time, but each Alert is sent to only one Emergency contact at the time of activation.
Implementation: alert.contact_id is a foreign key referencing contact.id
Example:

Contact ID	Contact Name	Alert ID	Timestamp
C001	Jane Doe	A001	2026-01-01 12:35:00
C001	Jane Doe	A002	2026-01-01 14:42:00
C002	Bob Smith	A003	2026-01-02 13:25:00


SoleSignal – Entity Relationship Diagram
Domain Model · Four entities, five relationships
users
PK id INTEGER
name TEXT
email TEXT UNIQUE
phone_number TEXT
password_hash TEXT
is_verified BOOLEAN
last_login TIMESTAMP?
created_at TIMESTAMP
sensors
PK id INTEGER
sensor_id TEXT UNIQUE
FK user_id INTEGER
is_paired BOOLEAN
is_calibrating BOOLEAN
last_connected TIMESTAMP?
created_at TIMESTAMP
1:1
1:M
1:M
1:M
1:M
emergency_contacts
PK id INTEGER
FK user_id INTEGER
name TEXT
phone_number TEXT
is_valid BOOLEAN
created_at TIMESTAMP
alerts
PK id INTEGER
FK user_id INTEGER
FK sensor_id INTEGER
FK contact_id INTEGER
gps_latitude FLOAT?
gps_longitude FLOAT?
loc_available BOOLEAN
timestamp TIMESTAMP
delivery_status TEXT
retry_count INTEGER
PK = Primary Key FK = Foreign Key ? = Nullable || = one and only one |< = one or many

Database Schema

To manage all the backend data for SoleSignal, I have selected a relational SQL database, specifically PostgreSQL. This works well because the system's core entities are all strongly connected through foreign key relationships. The data is structured and relational by nature. PostgreSQL helps keep these connections organized and enforces data integrity through constraints. It is also open source and widely supported, making it a practical choice for both development and future scaling.

Important notes:

- The delivery_status field uses a CHECK constraint with three states: pending, delivered, and failed to map to NFR-5's retry logic.
- For GPS coordinates, the Prisma schema uses Float (PostgreSQL DOUBLE PRECISION). The CREATE TABLE statements below show NUMERIC(9,6) as a conceptual reference; behavior is equivalent for this application's precision needs.
- The schema is managed by Prisma ORM in production. The source of truth is backend/prisma/schema.prisma. The CREATE TABLE statements below are provided as a conceptual reference. Migrations are applied using npx prisma migrate deploy.

CREATE TABLE Statements

users

```
CREATE TABLE users (
        id SERIAL PRIMARY KEY,
        name TEXT NOT NULL,
        email TEXT UNIQUE,
        phone_number TEXT UNIQUE,
        password_hash TEXT NOT NULL,
        is_verified BOOLEAN DEFAULT FALSE,
        last_login TIMESTAMP,
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Access Roles:

- User can read and update their own record (e.g. editing profile info)
- Backend System can read to authenticate and write during registration
- Admin has read access for account management and auditing

sensors

```
CREATE TABLE sensors (
id SERIAL PRIMARY KEY,
sensor_id TEXT UNIQUE NOT NULL,
user_id INTEGER REFERENCES users(id),
is_paired BOOLEAN DEFAULT FALSE,
```

```
is_calibrating BOOLEAN DEFAULT FALSE,
last_connected TIMESTAMP,
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Access Roles:

- Users can read their own paired sensor status
- Backend System can read and write during pairing, unpairing, and connection status updates
- Admin has read access for device management and auditing

Note: When a sensor record is deleted, all associated alert records are cascade deleted via the database foreign key constraint (onDelete: Cascade)

contacts (Emergency Contacts)

```
CREATE TABLE contacts (
        id SERIAL PRIMARY KEY,
        user_id INTEGER REFERENCES users(id),
        name TEXT NOT NULL,
        phone_number TEXT NOT NULL,
        is_valid BOOLEAN DEFAULT FALSE,
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Access Roles:

- User can read, write, and delete their own emergency contacts
- Backend System can read to retrieve contact information at time of alert delivery
- Admin has read access for auditing purposes

alerts

```
CREATE TABLE alerts (
        id SERIAL PRIMARY KEY,
        user_id INTEGER REFERENCES users(id),
        sensor_id INTEGER REFERENCES sensors(id),
        contact_id INTEGER REFERENCES contacts(id),
        gps_latitude NUMERIC(9,6),
        gps_longitude NUMERIC(9,6),
        location_available BOOLEAN DEFAULT TRUE,
        timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        delivery_status TEXT CHECK (delivery_status IN ('pending', 'delivered', 'failed')),
        retry_count INTEGER DEFAULT 0
);
```

Access Roles:

- User can read their own alert history
- Backend System can read and write to create alert records and update delivery status and retry count
- Admin has read access for auditing purposes

API Specification and REST API Design

The backend of the SoleSignal system is designed using REST architecture. Each REST API is built around an entity (users, sensors, contacts, alerts) and each operation uses a specific HTTP method to perform an action on that resource. For example, POST /users registers a new user account, while GET /sensors/{id} retrieves the status of a paired sensor. These APIs serve as the communication layer between the mobile frontend and the backend. The API receives data in JSON format, validates it, performs any necessary logic or database operations, and returns structured responses also in JSON.

Endpoint Overview Table

Method	URI	Operation Name	Use Case
POST	/users/register	registerUser()	UC-1: Register User Account
POST	/users/login	loginUser()	UC-3: Manage User profile
PATCH	/users/{id}	updateUserProfile()	UC-3: Manage User profile
POST	/users/logout	logoutUser()	UC-3: Manage User profile
POST	/sensors/pair	pairSensor()	UC-2: Pair Sensor with User Account
GET	/sensors/{id}	getSensorStatus()	UC-2: Pair Sensor with User Account
GET	/sensors/me	getMySensor()	UC-2: Pair Sensor with User Account
DELETE	/sensors/me	unpairSensor()	UC-2: Pair Sensor with User Account
POST	/contacts	addContact()	UC-4: Manage Emergency Contacts
GET	/contacts	getContacts()	UC-4: Manage Emergency Contacts
PATCH	/contacts/{id}	updateContact()	UC-4: Manage Emergency Contacts
DELETE	/contacts/{id}	deleteContact()	UC-4: Manage Emergency Contacts
POST	/alerts	sendAlert()	UC-6: Send Emergency Alert
GET	/alerts/{id}	getAlertStatus()	UC-6: Send Emergency Alert

registerUser()

Field	Description
Operation Name	registerUser()
Resource	users
HTTP Method	POST
URI	/users/register
User Role	Unauthenticated User
Use Case ID	UC-1: Register User Account
Input Arguments	JSON: { "name": string, "email": string, "phone_number": string, "password": string }
Input Validation	name: required, non-empty string. email or phone_number: at least one required, valid format. password: required, minimum 8 characters
Return Data	JSON: { "message": "Registration successful", "user_id": integer }
HTTP Status Codes	201 Created: account successfully created. 400 Bad Request: missing or invalid fields. 409 Conflict: email or phone number already registered. 422 Unprocessable Content: data is not processable. 500 Internal Server Error: database or system issue
Third Party API?	No

Example Request:

```
POST /users/register
Content-Type: application/json
{
  "name": "Jane Doe",
  "email": "janedoe@email.com",
  "phone_number": "7321234567",
  "password": "securepassword123"
}
```

Example Response:

```
{
  "message": "Registration successful",
  "user_id": 1
}
```

Example Failure (Email Already Registered):

```
HTTP/1.1 409 Conflict
```

```
Content-Type: application/json
{
  "error": "An account with this email already exists"
}
```

loginUser()

Field	Description
Operation Name	loginUser()
Resource	users
HTTP Method	POST
URI	/users/login
User Role	Registered User
Use Case ID	UC-3: Manage User Profile
Input Arguments	JSON: { "email": string, "password": string }
Input Validation	email: required, valid format. password: required
Return Data	JSON: { "message": "Login successful", "token": string }
HTTP Status Codes	200 OK: login successful. 400 Bad Request: missing fields. 401 Unauthorized: invalid credentials. 422 Unprocessable Content: data is not processable
Third Party API?	No

Example Request:

```
POST /users/login
Content-Type: application/json
{
  "email": "janedoe@email.com",
  "password": "securepassword123"
}
```

Example Response:

```
{
  "message": "Login successful",
  "token": "eyJhbGciOiJIUzI1NiIsInR5"
}
```

Example Failure (Invalid Credentials):

```
HTTP/1.1 401 Unauthorized
Content-Type: application/json
```

```
{
  "error": "Invalid email or password"
}
```

updateUserProfile()

Field	Description
Operation Name	updateUserProfile()
Resource	users
HTTP Method	PATCH
URI	/users/{id}
User Role	Registered User
Use Case ID	UC-3: Manage User Profile
Input Arguments	Path parameter: user ID (integer). JSON: { "name": string, "email": string, "phone_number": string }
Input Validation	id: required, must be a valid user ID. All fields optional but at least one must be present. email/phone_number: valid format if provided
Return Data	JSON: { "message": "Profile updated successfully" }
HTTP Status Codes	200 OK: update successful. 400 Bad Request: invalid input. 404 Not Found: user not found. 422 Unprocessable Content: data is not processable. 500 Internal Server Error: database or system failure
Third Party API?	No

Example Request:

```
PATCH /users/1
Content-Type: application/json
Authorization: Bearer <token>
{
  "name": "Jane Smith"
}
```

Example Response:

```
{
  "message": "Profile updated successfully"
}
```

Example Failure (User Not Found):

```
HTTP/1.1 404 Not Found
Content-Type: application/json
{
  "error": "User with ID 99999 not found"
}
```

logoutUser()

Field	Description
Operation Name	logoutUser()
Resource	users
HTTP Method	POST
URI	/users/logout
User Role	Registered User
Use Case ID	UC-3: Manage User Profile
Input Arguments	None (token read from Authorization header)
Input Validation	Authorization header: required, must contain a valid Bearer token
Return Data	JSON: { "message": "Logged out successfully" }
HTTP Status Codes	200 OK: logout successful. 401 Unauthorized: missing or invalid token. 500 Internal Server Error: server-side failure
Third Party API?	No

Example Request:

```
POST /users/logout
Authorization: Bearer <token>
```

Example Response:

```
{
  "message": "Logged out successfully”
}
```

Example Failure (Invalid Credentials):

```
HTTP/1.1 401 Unauthorized
Content-Type: application/json
{
  "error": "Authentication Token Required"
}
```

pairSensor()

Field	Description
Operation Name	pairSensor()
Resource	sensors
HTTP Method	POST
URI	/sensors/pair
User Role	Registered User
Use Case ID	UC-2: Pair Sensor with User Account
Input Arguments	JSON: { "sensor_id": string }
Input Validation	sensor_id: required, must match a registered hardware identifier. Sensor must not already be paired to another account. user_id is extracted from the JWT token, not the request body.
Return Data	JSON: { "message": "Sensor paired successfully", "sensor_id": string, "id": integer }
HTTP Status Codes	200 OK: sensor paired successfully. 400 Bad Request: missing or invalid fields. 409 Conflict: sensor already paired to another account. 422 Unprocessable Content: data is not processable. 500 Internal Server Error: database or system failure
Third Party API?	No

Example Request:

```
POST /sensors/pair
Content-Type: application/json
Authorization: Bearer <token>
{
  "sensor_id": "SS-00142"
}
```

Example Response:

```
{
  "message": "Sensor paired successfully",
  "sensor_id": "SS-00142",
 “id”: 1
}
```

Example Failure (Sensor Already Paired):

```
HTTP/1.1 409 Conflict
```

```
Content-Type: application/json
{
  "error": "This sensor is already paired to another account"
}
```

getSensorStatus()

Field	Description
Operation Name	getSensorStatus()
Resource	sensors
HTTP Method	GET
URI	/sensors/{id}
User Role	Registered User
Use Case ID	UC-2: Pair Sensor with User Account
Input Arguments	Path parameter: sensor ID (integer)
Input Validation	id: required, must be a valid sensor ID belonging to the authenticated user
Return Data	JSON: { "sensor_id": string, "is_paired": boolean, "is_calibrating": boolean, "last_connected": timestamp }
HTTP Status Codes	200 OK: sensor status retrieved. 404 Not Found: sensor not found. 422 Unprocessable Content: data is not processable. 500 Internal Server Error: database or system failure
Third Party API?	No

Example Request:

```
GET /sensors/1
Authorization: Bearer <token>
```

Example Response:

```
{
  "sensor_id": "SS-00142",
  "is_paired": true,
  "is_calibrating": false,
  "last_connected": "2026-03-23T10:15:00Z"
}
```

getMySensor()

Field	Description

Operation Name	getMySensor()
Resource	sensors
HTTP Method	GET
URI	/sensors/me
User Role	Registered User
Use Case ID	UC-2: Pair Sensor with User Account
Input Arguments	User identified via JWT token
Input Validation	Authorization header required
Return Data	JSON: { "id": integer, "sensor_id": string, "is_paired": boolean, "is_calibrating": boolean, "last_connected": timestamp }
HTTP Status Codes	200 OK: sensor found. 404 Not Found: user has no paired sensor. 401 Unauthorized: missing or invalid token
Third Party API?	No

Example Request:

```
GET /sensors/me
Authorization: Bearer <token>
```

Example Response:

```
{
 "id": 1,
 "sensor_id": "B70FA814-...",
 "is_paired": true,
 "is_calibrating": false,
 "last_connected": "2026-03-30T14:00:00Z"
}
```

unpairSensor()

Field	Description
Operation Name	unpairSensor()
Resource	sensors
HTTP Method	DELETE
URI	/sensors/me
User Role	Registered User
Use Case ID	UC-2: Pair Sensor with User Account
Input Arguments	User identified via JWT token
Input Validation	Authorization header required
Return Data	JSON: { "message": “Sensor unpaired successfully” }
HTTP Status Codes	200 OK: sensor unpaired and deleted.

	404 Not Found: user has no sensor to unpair. 401 Unauthorized: missing or invalid token
Third Party API?	No

Example Request:

```
DELETE /sensors/me
Authorization: Bearer <token>
```

Example Response:

```
{
  "message": “Sensor unpaired successfully”
}
```

addContact()

Field	Description
Operation Name	addContact()
Resource	contacts
HTTP Method	POST
URI	/contacts
User Role	Registered User
Use Case ID	UC-4: Manage Emergency Contacts
Input Arguments	JSON: { "name": string, "phone_number": string }
Input Validation	name: required, non-empty string. phone_number: required, valid format and SMS capability check
Return Data	JSON: { "message": "Contact added successfully", "contact_id": integer }
HTTP Status Codes	201 Created: contact successfully added. 400 Bad Request: missing or invalid fields. 422 Unprocessable Content: data is not processable. 500 Internal Server Error: database or system failure
Third Party API?	No

Example Request:

```
POST /contacts
Content-Type: application/json
Authorization: Bearer <token>
{
  "name": "John Doe",
  "phone_number": "7329876543"
```

```
}
```

Example Response:

```
{
  "message": "Contact added successfully",
  "contact_id": 1
}
```

Example Failure (Invalid Phone Number):

```
HTTP/1.1 400 Bad Request
Content-Type: application/json
{
  "error": "Phone number format is invalid"
}
```

getContacts()

Field	Description
Operation Name	getContacts()
Resource	contacts
HTTP Method	GET
URI	/contacts
User Role	Registered User
Use Case ID	UC-4: Manage Emergency Contacts
Input Arguments	User identified via JWT token
Input Validation	user_id: required, must be a valid user ID
Return Data	JSON: { "contacts": [{ "id": integer, "name": string, "phone_number": string, "is_valid": boolean }, ...] }
HTTP Status Codes	200 OK: contacts retrieved. 400 Bad Request: invalid or missing user_id. 500 Internal Server Error: database or system failure.
Third Party API?	No

Example Request:

```
GET /contacts
Authorization: Bearer <token>
```

Example Response:

```
{
  "contacts": [
  { "id": 1, "name": "John Doe", "phone_number": "7329876543", "is_valid": true }
  ]
```

```
}
```

updateContact()

Field	Description
Operation Name	updateContact()
Resource	contacts
HTTP Method	PATCH
URI	/contacts/{id}
User Role	Registered User
Use Case ID	UC-4: Manage Emergency Contacts
Input Arguments	Path parameter: contact ID (integer). JSON: { "name": string, "phone_number": string }
Input Validation	id: required, valid contact ID belonging to the authenticated user. At least one field must be present. phone_number: valid format if provided
Return Data	JSON: { "message": "Contact updated successfully" }
HTTP Status Codes	200 OK: update successful. 400 Bad Request: invalid input. 404 Not Found: contact not found. 500 Internal Server Error: database or system failure
Third Party API?	No

Example Request:

```
PATCH /contacts/1
Content-Type: application/json
Authorization: Bearer <token>
{
  "phone_number": "7321112222"
}
```

Example Response:

```
{
  "message": "Contact updated successfully"
}
```

deleteContact()

Field	Description
Operation Name	deleteContact()
Resource	contacts

HTTP Method	DELETE
URI	/contacts/{id}
User Role	Registered User
Use Case ID	UC-4: Manage Emergency Contacts
Input Arguments	Path parameter: contact ID (integer)
Input Validation	id: required, valid contact ID belonging to the authenticated user
Return Data	JSON: { "message": "Contact deleted successfully" }
HTTP Status Codes	200 OK: deletion successful. 404 Not Found: contact not found. 500 Internal Server Error: database or system failure
Third Party API?	No

Example Request:

```
DELETE /contacts/1
Authorization: Bearer <token>
```

Example Response:

```
{
  "message": "Contact deleted successfully"
}
```

sendAlert()

Field	Description
Operation Name	sendAlert()
Resource	alerts
HTTP Method	POST
URI	/alerts
User Role	Backend System (triggered by validated tap pattern)
Use Case ID	UC-6: Send Emergency Alert
Input Arguments	JSON: { "sensor_id": integer, "contact_id": integer, "gps_latitude": decimal, "gps_longitude": decimal, "location_available": boolean }
Input Validation	sensor_id, contact_id: required, valid IDs. user_id is extracted from the JWT token. gps_latitude/gps_longitude: required if location_available is true. location_available: required boolean
Return Data	JSON: { "message": "Alert sent successfully", "alert_id": integer, "delivery_status": string }
HTTP Status Codes	201 Created: alert created and sent. 400 Bad Request: missing or invalid fields. 404 Not Found: user, sensor, or contact not found. 500 Internal Server Error: database or Textbelt failure. 503 Service Unavailable: Textbelt unavailable, retry scheduled

Third Party API?	Yes, it triggers Textbelt SMS delivery to the emergency contact

Example Request:

```
POST /alerts
Content-Type: application/json
Authorization: Bearer <token>
{
  "sensor_id": 1,
  "contact_id": 1,
  "gps_latitude": 40.500000,
  "gps_longitude": -74.450000,
  "location_available": true
}
```

Example Response:

```
{
  "message": "Alert sent successfully",
  "alert_id": 1,
  "delivery_status": "delivered"
}
```

Example Failure (Textbelt Unavaliable):

```
HTTP/1.1 503 Service Unavailable
Content-Type: application/json
{
  "error": "SMS delivery failed. Retrying delivery, attempt 1 of 3."
}
```

getAlertStatus()

Field	Description
Operation Name	getAlertStatus()
Resource	alerts
HTTP Method	GET
URI	/alerts/{id}
User Role	Registered User
Use Case ID	UC-6: Send Emergency Alert
Input Arguments	Path parameter: alert ID (integer)
Input Validation	id: required, must be a valid alert ID belonging to the authenticated user

Return Data	JSON: { "alert_id": integer, "delivery_status": string, "retry_count": integer, "timestamp": timestamp }
HTTP Status Codes	200 OK: status retrieved. 404 Not Found: alert not found. 500 Internal Server Error: database or system failure
Third Party API?	No

Example Request:

```
GET /alerts/1
Authorization: Bearer <token>
```

Example Response:

```
{
 "alert_id": 1,
 "delivery_status": "delivered",
 "retry_count": 0,
 "timestamp": "2026-03-23T10:22:00Z"
}
```

Server Side Routing of HTTP Requests

The following describes how each HTTP request flows through the SoleSignal backend, from the frontend service call through the controller and into the database. Each endpoint follows the same general pattern: the frontend sends a request, a route definition maps it to a controller function, the controller validates the input and performs the necessary logic, and the database is queried or updated accordingly.

Auth Middleware

All protected endpoints pass through middleware/auth.ts before the controller runs. The middleware reads the Authorization header, extracts the Bearer token, verifies the token signature using the JWT_SECRET environment variable, and attaches req.user = { user_id, email } to the request. If the token is missing, malformed, or expired, the middleware returns 401 immediately. Controllers never manually verify the token, rather they simply read req.user.user_id to identify the authenticated user.

registerUser()

Frontend Call: The registration service sends a POST request to /users/register with the user's name, email or phone number, and password.
Route Definition: The auth router maps POST /users/register to the registerUser() controller function.
Controller Logic: The controller validates that at least one of email or phone number is present and correctly formatted, checks that the password meets the minimum length requirement, verifies that no existing account shares the same email or phone number, hashes the password securely, and inserts the new user record.
Database Interaction:

```
INSERT INTO users (name, email, phone_number, password_hash)
VALUES (...)
```

Response: Returns 201 with the new user_id on success. Returns 409 if the email or phone number is already registered.

loginUser()

Frontend Call: The authentication service sends a POST request to /users/login with the user's email and password.
Route Definition: The auth router maps POST /users/login to the loginUser() controller function.
Controller Logic: The controller retrieves the user record by email, compares the provided password against the stored hash, and if valid, generates and returns a session token.
Database Interaction:

```
SELECT id, password_hash FROM users
WHERE email = {email}
```

Response: Returns 200 with a session token on success. Returns 401 if credentials are invalid.

updateUserProfile()

Frontend Call: The user service sends a PATCH request to /users/{id} with the fields to be updated.
Route Definition: The user router maps PATCH /users/{id} to the updateUserProfile() controller function.
Controller Logic: The controller validates the user ID, confirms the requesting user matches the target record, validates any updated fields, and applies the changes.
Database Interaction:

```
UPDATE users
SET name = {name}, email = {email}, phone_number = {phone_number}
WHERE id = {id}
```

Response: Returns 200 on success. Returns 404 if the user ID does not exist.

logoutUser()

Frontend Call: The authentication service sends a POST request to /users/logout with the Bearer token included in the Authorization header.
Route Definition: The auth router maps POST /users/logout to the logoutUser() controller function.
Controller Logic: The controller validates that a Bearer token is present and well-formed. Since JWTs are stateless and the MVP does not maintain a server-side token blacklist, the controller does not perform a database write. The response instructs the client to delete the token from secure storage. If the token is missing or malformed, a 401 is returned.
Database Interaction: None. Logout is handled entirely on the client side by deleting the stored token. No database query is required.
Response: Returns 200 with a confirmation message on success. Returns 401 if the Authorization header is missing or the token is malformed.

pairSensor()

Frontend Call: The sensor service sends a POST request to /sensors/pair with the sensor_id. The user_id is extracted from the JWT token by the auth middleware.
Route Definition: The sensor router maps POST /sensors/pair to the pairSensor() controller function.
Controller Logic: The controller reads sensor_id from req.body and user_id from req.user.user_id (JWT token). It verifies the sensor is not already paired to another account, then links the sensor to the user and sets is_paired to true.
Database Interaction:

```
UPDATE sensors
SET user_id = {user_id}, is_paired = TRUE
WHERE sensor_id = {sensor_id}
```

Response: Returns 200 with the sensor_id on success. Returns 409 if the sensor is already paired to another account.

getSensorStatus()

Frontend Call: The sensor service sends a GET request to /sensors/{id}.
Route Definition: The sensor router maps GET /sensors/{id} to the getSensorStatus() controller function.
Controller Logic: The controller validates the sensor ID, confirms the sensor belongs to the authenticated user, and retrieves the current sensor status.
Database Interaction:

```
SELECT sensor_id, is_paired, is_calibrating, last_connected
FROM sensors
WHERE id = {id}
```

Response: Returns 200 with the sensor status object. Returns 404 if the sensor is not found.

getMySensor()

Frontend Call: The sensor service sends a GET request to /sensors/me.
Route Definition: The sensor router maps GET /sensors/me to the getMySensor() controller function.
Controller Logic: The controller reads user_id from req.user.user_id (JWT token) and retrieves the sensor record where user_id matches.
Database Interaction:

```
SELECT id, sensor_id, is_paired, is_calibrating, last_connected
FROM sensors
WHERE user_id = {user_id}
```

Response: Returns 200 with the sensor object. Returns 404 if no sensor is found for this user.

unpairSensor()

Frontend Call: The sensor service sends a DELETE request to /sensors/me.
Route Definition: The sensor router maps DELETE /sensors/me to the unpairSensor() controller function.
Controller Logic: The controller reads user_id from req.user.user_id (JWT token), finds the sensor record belonging to that user, and deletes it. Associated alert records are cascade-deleted by the database foreign key constraint.
Database Interaction:

```
DELETE FROM sensors
WHERE user_id = {user_id}
```

Response: Returns 200 on success. Returns 404 if no sensor is found for this user.

addContact()

Frontend Call: The contact service sends a POST request to /contacts with the name and phone_number. The user_id is extracted from the JWT token by the auth middleware.
Route Definition: The contact router maps POST /contacts to the addContact() controller function.
Controller Logic: The controller reads user_id from req.user.user_id (JWT token), validates the phone number format and SMS capability, and inserts the new contact record linked to the user.
Database Interaction:

```
INSERT INTO contacts (user_id, name, phone_number, is_valid)
VALUES (...)
```

Response: Returns 201 with the new contact_id on success. Returns 400 if the phone number format is invalid.

getContacts()

Frontend Call: The contact service sends a GET request to /contacts. The user_id is extracted from the JWT token by the auth middleware.
Route Definition: The contact router maps GET /contacts to the getContacts() controller function.
Controller Logic: The controller reads user_id from req.user.user_id (JWT token) and retrieves all contacts associated with that user.
Database Interaction:

```
SELECT id, name, phone_number, is_valid
FROM contacts
WHERE user_id = {user_id}
```

Response: Returns 200 with a wrapped contacts array. Returns an empty array if no contacts exist.

updateContact()

Frontend Call: The contact service sends a PATCH request to /contacts/{id} with the fields to be updated.
Route Definition: The contact router maps PATCH /contacts/{id} to the updateContact() controller function.
Controller Logic: The controller validates the contact ID, confirms the contact belongs to the authenticated user, validates any updated fields, and applies the changes.
Database Interaction:

```
UPDATE contacts
SET name = {name}, phone_number = {phone_number}
WHERE id = {id}
```

Response: Returns 200 on success. Returns 404 if the contact is not found.

deleteContact()

Frontend Call: The contact service sends a DELETE request to /contacts/{id}.

Route Definition: The contact router maps DELETE /contacts/{id} to the deleteContact() controller function.
Controller Logic: The controller validates the contact ID, confirms the contact belongs to the authenticated user, and removes the record.
Database Interaction:

```
DELETE FROM contacts
WHERE id = {id}
```

Response: Returns 200 on success. Returns 404 if the contact is not found.

sendAlert()

Frontend Call: The alert service sends a POST request to /alerts with the sensor_id, contact_id, GPS coordinates, and location_available flag. The user_id is extracted from the JWT token by the auth middleware.
Route Definition: The alert router maps POST /alerts to the sendAlert() controller function.
Controller Logic: The controller validates all required IDs, assembles the alert package including user identity, sensor ID, GPS location if available, and timestamp, inserts the alert record with a delivery_status of pending, and forwards the alert package to the Textbelt API for SMS delivery. If delivery succeeds, delivery_status is updated to delivered. If delivery fails, the system retries up to 3 times before setting delivery_status to failed and notifying the user.
Database Interaction:

```
INSERT INTO alerts (user_id, sensor_id, contact_id, gps_latitude,
gps_longitude, location_available, timestamp, delivery_status, retry_count)
VALUES (...)

UPDATE alerts
SET delivery_status = {status}, retry_count = {count}
WHERE id = {alert_id}
```

Response: Returns 201 with alert_id and delivery_status on success. Returns 503 if Textbelt is unavailable and retries are scheduled.

getAlertStatus()

Frontend Call: The alert service sends a GET request to /alerts/{id}.
Route Definition: The alert router maps GET /alerts/{id} to the getAlertStatus() controller function.
Controller Logic: The controller validates the alert ID, confirms the alert belongs to the authenticated user, and retrieves the current delivery status.
Database Interaction:

```
SELECT alert_id, delivery_status, retry_count, timestamp
FROM alerts
WHERE id = {id}
```

Response: Returns 200 with the alert status object. Returns 404 if the alert is not found.

Authentication and Session Management

SoleSignal uses JSON Web Token (JWT) based authentication to secure all protected endpoints. When a user successfully logs in, the server generates a signed JWT and returns it to the mobile application. The client stores this token and includes it in the authorization header of every subsequent request. The server validates the token on each request before allowing access to protected resources.

Token Generation

When a user successfully authenticates via loginUser(), the server generates a JWT containing the following payload:

```
{
    "user_id": 1,
    "email": "janedoe@email.com",
    "iat": 1742000000,
    "exp": 1742086400
}
```

Where:

- user_id is used to scope all database queries to the authenticated user
- email is used to identify the account
- iat is issued at timestamp
- exp is the expiry timestamp (default 24 hours from issue timestamp)

The token is signed using a secret key stored securely in the server's environment variables and never exposed to the client.

Token Storage

On the mobile device, the JWT is stored in the device's secure storage (iOS Keychain), managed via react-native-keychain. It must not be stored in plain text or in any location accessible to other applications. The token persists across sessions until it expires, or the user explicitly logs out.

Token Usage

Every protected endpoint requires the token to be passed in the Authorization header as follows:
Authorization: Bearer <token>

If the token is missing or malformed, the server returns:

```
HTTP/1.1 401 Unauthorized
{
“error”: “Authentication token required”
}
```

If the token has expired, the server returns:
HTTP/1.1 401 Unauthorized
{
"error": "Token expired, please log in again"
}

Token Expiry and Renewal

Tokens expire 24 hours after issue. When a token expires, the user is required to log in again to obtain a new token. There is no refresh token mechanism in the MVP, so reauthentication is the only renewal path.

Protected vs. Unprotected Endpoints

Endpoint	Protected
POST /users/register	No
POST /users/login	No
PATCH /users/{id}	Yes
POST /users/logout	Yes
POST /sensors/pair	Yes
GET /sensors/{id}	Yes
GET /sensors/me	Yes
DELETE /sensors/me	Yes
POST /contacts	Yes
GET /contacts	Yes
PATCH /contacts/{id}	Yes
DELETE /contacts/{id}	Yes
POST /alerts	Yes
GET /alerts/{id}	Yes

Ownership Enforcement

In addition to verifying the token, the server enforces ownership on all resource specific endpoints. For example, a user with user_id '1' cannot access or modify a contact or alert belonging to user_id '2'. The controller extracts the user_id from the validated token and uses it to scope all database queries, rather than trusting the user_id passed in the request body.

Logout Procedure

When a user logs out, the token is deleted from the device's secure storage. Since JWTs are stateless, the server does not maintain a blacklist of invalidated tokens in the MVP. The token simply becomes unused on the client side until it naturally expires.